

Exercícios – Objetivo 103.4 – Respostas

1. `$ cc -Wall -o prog prog.c 2>&1 | tee gcc.log | grep -c warning:`

2. `$ cmd1 | cmd2 2>&1 | cmd3 | cmd4`

3. `$ cmd1 | cmd2 2>&1 >/dev/null | cmd3 | cmd4`

Observe que é preciso redirecionar `stderr` **ANTES** de redirecionar `stdout`.

4. O redirecionamento de `stdout` cria o arquivo `a` (com zero bytes) antes de `cp` ser executado. Assim, o arquivo vazio `a` é copiado para o arquivo `b`.

5. A explicação é a mesma do exercício anterior: o redirecionamento zera o arquivo `a`, que é copiado por `cp` sobrescrevendo o arquivo `b` (que fica com conteúdo zerado).

6. `$ unset var1 ### garante que $var1 seja indefinida`

```
$ od -c <<<$var1
```

```
00000000 \n
```

```
00000001
```

Nenhum erro é indicado, e o comando recebe uma linha em branco (apenas um LF, ou `\n`).

7. `$ find . -maxdepth 1 -name \*.py -type f -print0 | xargs -0 sha256sum`

8. (a) O primeiro comando executa um `grep` para cada arquivo, ao passo que o segundo comando executa um único `grep` passando todos os arquivos como argumentos.

(b) Abaixo são mostradas duas soluções possíveis. A primeira controla a passagem de argumentos para `grep`, e a segunda filtra a saída do comando.

```
$ find . -name \*.c | xargs -n 1 grep -c \#include
```

```
$ find . -name \*.c | xargs grep -c \#include | cut -d: -f2
```

9. `$ find /lib/modules/$(uname -r) -name \*.ko -type f -print | wc -l`

Como `wc` conta apenas as linhas, não é necessário se preocupar com nomes de arquivos contendo espaços na saída de `find`.